MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

Prompt Engineering for Agents

How words become control flow: constitutions, format contracts, and token budgets for long-running agents.

Unit 04 · Part 1 of 2 · Session 04 · Week 4
Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **Where we are** from tools to the words that steer them
2. **The system prompt as constitution** role, capabilities, constraints, tool policy
3. **Format and reasoning control** few-shot, chain-of-thought, structured output
4. **Prompting the long run** system vs. appended context, window budgeting
5. **Prompt-as-code** versioning, regression tests, failure patterns

Where we are

Sessions 1-3 built the machinery. Today: the text that controls it.

Session 02 — the loop — model call, tool call, observation, repeat

Session 03 — tools — function calling and MCP define what it CAN do

Today — the prompt steers what it will do; runtime code enforces action limits

- each loop iteration is one big text completion
- prompts are one control surface — tools, permissions, decoding and validation are others

ONE MECHANISM

Everything the agent knows at step k is a single token sequence. Prompt engineering decides what is in it — and where. Enforcement lives in code.

1

The system prompt as constitution

Message roles, and the document that defines the agent

Message roles: who is speaking?

Chat models see a list of typed messages, serialized into one token stream

system — operator-authored policy; persists only as the application re-supplies it each call

user — the task and follow-ups; often untrusted in products

assistant — the model's own outputs, including tool-call requests

tool — results the runtime appends after executing a call

- a chat template turns roles into special tokens

NOT A FIREWALL

Roles are trained conventions inside one token stream — useful structure, not a security boundary. Session 13 shows how attackers exploit exactly this.

A realistic system prompt

An example constitution for a source-evaluation assistant (Lab 04) — short, operational, checkable

system_prompt.md

```
You are a source-evaluation assistant. You assess one
fetched page at a time for a given research brief.

## Capabilities
Input: page text with URL, author, date. No browsing.

## Constraints
- Output one JSON object: url, reliability, key_claims, flags.
- reliability = rubric score 0-1: primary evidence, authority,
  provenance, currency vs. the claim. Not a probability.
- List only claims made in the text. Unknown: "not found".
- Missing author or date: add a flag, do not reject.

## Tool policy
- One assessment per source; refuse out-of-scope tasks.
```

The four pillars of an agent constitution

Every durable system prompt answers these four questions

Role

Who the agent is, for whom it works, and the scope of its judgement. Sets register, persona stability, and what counts as a finished task.

Capabilities

What each tool is FOR, beyond its raw schema: when to search versus fetch, what good arguments look like, how results should be used.

Constraints

Intended policy regardless of the task: citation duties, forbidden actions, honesty rules, the output contract. The prompt states them; runtime validation enforces them.

Tool policy

Budgets and etiquette for tool use: call limits, retry rules, which actions are irreversible and need human approval before execution.

Vague intent vs. operational instruction

Instructions survive long runs only if the model can check them

Vague — decays under pressure

- "Be concise."
- "Use tools when helpful."
- "Don't hallucinate."
- "Format the report nicely."
- "Be careful with sources."

Operational — checkable

- "Max 300 words per section."
- "Search for time-sensitive or consequential claims."
- "Fetch before citing; write 'not found' when unsure."
- "Output must parse as Markdown + [n] cites."
- "Missing author or date: flag it as uncertainty."

2

Format and reasoning control

Few-shot examples, chain-of-thought, structured output

Few-shot prompting: show, don't describe

In-context learning as a format-control instrument (Brown et al., 2020)

Mechanism — the model continues patterns in its context — no weight updates

Format control — one worked example often pins output shape better than prose — verify on your model

Dosage — 1-3 examples sufficed in GPT-3-era studies; ablate zero- vs. few-shot on the target model

Pitfall — models copy content and biases of examples, not only their form

In agents — demonstrate one tool call or one report skeleton — not ten variants

Few-shot in the message list

A fabricated user/assistant pair demonstrates the contract

few_shot.py

```
messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    # one worked example pins the output format
    {"role": "user",
     "content": "Assess source: <article A>"},
    {"role": "assistant",
     "content": '{"url": "destatis.de", '
                '"reliability": 0.9, '  # rubric score
                '"key_claims": ["GDP grew 0.4%"]}'},
    # the real task follows the same pattern
    {"role": "user",
     "content": "Assess source: <article B>"},
]
```

Chain-of-thought: tokens buy reasoning

Intermediate steps as computation (Wei et al., 2022)

Mechanism — generated steps give the model working state and compute per answer

Evidence — few-shot CoT exemplars lifted arithmetic and symbolic reasoning in 2022 models (Wei et al., 2022)

Scale-dependent — in those models gains appeared only at scale; small models wrote fluent, wrong chains

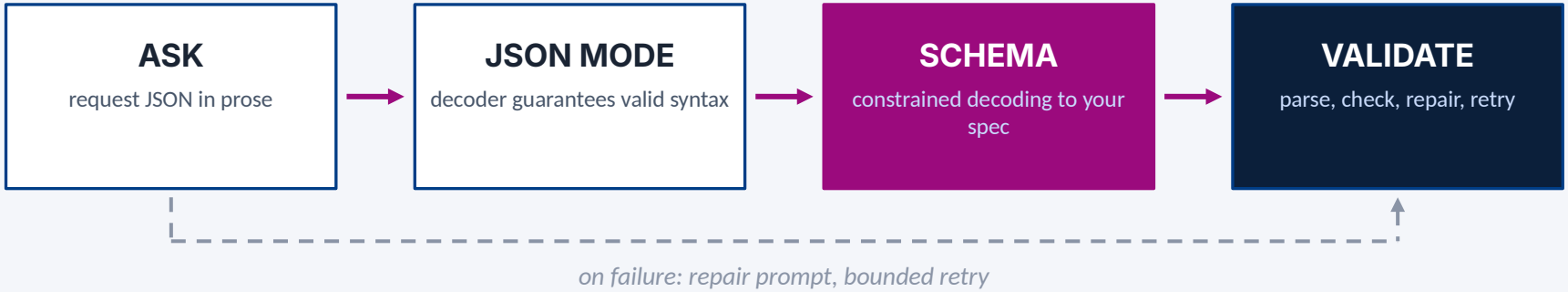
Zero-shot CoT — "Let's think step by step" alone helped (Kojima et al., 2022) — same model generation

When it pays — multi-step problems; for lookups it adds cost and latency; extra reasoning tokens can also hurt

In agents — the ReAct 'thought' is CoT interleaved with actions; trained reasoners internalize it (S05) — re-test

The output-control ladder

From polite requests to grammar-level guarantees



- JSON mode guarantees syntax — not your fields, types, or semantics
- schema enforcement masks tokens that would violate the declared structure
- validation with bounded retries remains the last line of defense

Schema enforcement in practice

Declare the type once; the API constrains decoding to match (cloud example; Lab 04 uses Ollama's schema format)

report_schema.py

```
from pydantic import BaseModel, Field

class SourceNote(BaseModel):
    url: str
    reliability: float = Field(ge=0, le=1)
    key_claims: list[str]

resp = client.chat.completions.parse(  # cloud
    model=MODEL, messages=messages,
    response_format=SourceNote)
choice = resp.choices[0]; msg = choice.message
if choice.finish_reason != "stop":  # cut off
    raise RuntimeError(choice.finish_reason)
if msg.refusal or msg.parsed is None:
    raise RefusalError(msg.refusal)
note = msg.parsed  # typed SourceNote
```

Guarantee — schema-conformant for supported schemas and complete, non-refusal replies

Not guaranteed — values true; reliability is a rubric score

- refusal, truncation, API errors: own paths
- tool calls use the same machinery

3

Prompting the long run

What goes where when the context grows for 40 steps

System prompt vs. appended context

Two zones with different lifetimes — keep them apart

System — stable for the run: role, constraints, tool policy, output contract

Appended — everything that changes: task, tool results, working notes

Caching — providers cache the stable prefix — stability buys speed and cost

Decay — instructions buried mid-history fade; durable rules live in system

- do not restate the constitution every turn — it bloats and drifts

RULE OF THUMB

If it must still hold at step 40, keep it in the system prompt the application re-sends every call — and enforce it in code. The system role steers; it does not bind.

Budgeting the context window

An illustrative, disjoint split for a long search-read-write run — measure your real shares

Component	Content	Illustrative share
System prompt	constitution, output contract	3%
Tool definitions	JSON schemas for 5-15 tools	5%
Few-shot examples	one worked format example	2%
History (control)	steps, calls, notes; no evidence text	35%
Evidence text	fetched pages, retrieved passages	40%
Output reserve	headroom for answer and reasoning	15%

Sums to 100% by construction, not by measurement. Log real shares per step; model output and reasoning limits are budgeted separately (Session 08).

Position is not neutral

"Lost in the middle": long-context accuracy is U-shaped.

Liu et al. (TACL 2024, 2023-era models): facts are retrieved best from the start and end of the context, worst from the middle. "Rules first, live task last" is a heuristic to ablate on your model, not a law.

Tactics for staying inside the budget

Prompt-level moves; full memory architectures follow in Session 08

Reserve output space — leave headroom — truncation at the end kills the final answer

Trim tool results — cap page dumps; keep full text outside the window, reference it

Compact — summarize old steps into notes once a threshold is crossed

Just-in-time context — inject only what the next step needs, not all you have

Order deliberately — rules first, evidence middle, live task last — a heuristic; test placement on your model

Measure — log tokens per component per step; replace the illustrative budget with measured shares

4

Prompt-as-code and failure patterns

Engineering discipline for the most fragile file in the repo

Prompt-as-code: version it, test it, ship it

The prompt defines behavior — treat it like source, not like config prose

Version together — prompt text, model ID, and sampling parameters are one artifact

Templates — explicit slots for task and context — no string concatenation soup

Nonlocal effects — changing one sentence can silently break an unrelated case

Regression suite — fixed inputs, assertions on outputs, run on every prompt edit

Staged rollout — A/B or canary a new prompt version; keep instant rollback

- model upgrades are prompt regressions too — re-run the suite

A prompt contract test

Assert the contract, not the wording. Contract test only; it does not establish factual correctness

test_prompts.py

```
import json, pytest
from agent import run_task
from checks import claims, sources_resolve, supports
from fixtures import FAKE_REPORT  # cites [7]; no [7] entry
CASES = json.load(open("eval/cases.json"))

@pytest.mark.parametrize("case", CASES)
def test_report_contract(case):  # contract, not truth
    report = run_task(case["task"])
    found = set(claims(report))
    assert found and set(case["expected_claims"]) <= found
    assert sources_resolve(report)  # ids exist, URLs fetch
    for c in found:
        assert supports(c, report)  # cited passage backs c
def test_fabricated_citation_rejected():
    assert not sources_resolve(FAKE_REPORT)
```

Failure patterns and their fixes

Three drifts dominate agent prompt failures in practice

Failure	Symptom	Fix
Ignored instructions	rule buried mid-context is silently dropped	move to system; make checkable; validate output
Role drift	agent adopts tone or persona from fetched text	re-anchor role after compaction; delimit untrusted content
Format drift	JSON or Markdown degrades over long runs	schema enforcement; validate every step; bounded retry
Example overfitting	model copies few-shot content, not just form	neutral examples, clearly marked; vary them

Sclar et al. (2024): even formatting choices alone can swing accuracy by double digits.

Lab 04: repair a broken constitution

Gap-fill notebook, local Ollama model (qwen2.5:7b), offline corpus data/sources.json, 90-120 min

Parts B-C — diagnose a vague system prompt against the four pillars; rewrite it into checkable clauses

Part D — fabricated few-shot history; measure zero-shot vs. few-shot parse rates over repeated runs

Part E — climb the ladder: ASK, JSON mode, schema, validation with bounded repair retries

Part F — property-based regression harness: contract pass rates across temperatures and runs

Part G — ablate a pillar, change formatting only (Sclar et al.), probe example contamination

Report R1-R4 — operational rewrite, bounded retry code, failure compounding, pass rate vs. perfection

THE TASK

Assess one of six synthetic 'fetched pages' of very different quality at a time. No web access, no API keys — the whole lab runs locally.

TAKEAWAYS

What you should remember

- **Constitution —** the system prompt states role, capabilities, constraints, and tool policy — operational, not vague
- **Steer vs. enforce —** prompts steer behavior; runtime code enforces action limits and validates outputs
- **Show the format —** a worked example or an enforced schema beats prose — verify on the target model
- **Budget the window —** stable system prefix, trimmed history, output reserve; position effects are heuristics to test
- **Prompt-as-code —** versioned, contract-tested, canary-deployed — like any behavior-defining source

NEXT SESSION Reasoning models — and how they reshape agent design.

Reading & references

- Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. arXiv:2201.11903.
  The CoT paper — read sections 1-3 and the scaling analysis.
- Kojima, T. et al. (2022). Large Language Models are Zero-Shot Reasoners. NeurIPS 2022. arXiv:2205.11916.
  "Let's think step by step" — zero-shot CoT.
- Brown, T. et al. (2020). Language Models are Few-Shot Learners. NeurIPS 2020. arXiv:2005.14165.
  GPT-3: in-context learning without weight updates.
- Liu, N. F. et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. TACL 12. arXiv:2307.03172.
  U-shaped position effect in long contexts.
- Sclar, M. et al. (2024). Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design. ICLR 2024. arXiv:2310.11324.
  Format alone can swing benchmark accuracy dramatically.
- Anthropic (2024). Building Effective Agents. anthropic.com/engineering.
  Practical guidance on prompts and agent structure.
- Anthropic (2025). Effective Context Engineering for AI Agents. Engineering blog.
  System vs. appended context, compaction, just-in-time retrieval.